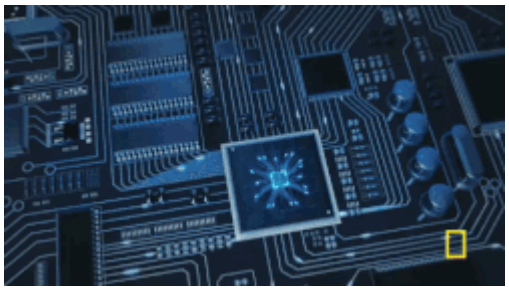
Boolesche Algebra

Parameter	Kursinformationen
Veranstaltung:	`Digitale Systeme / Eingebettete Systeme`
Semester:	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Boolesche Algebra, Axiome, Schaltfunktionen und technische Realisierung`
Link auf GitHub:	https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme/blob/master/02_BoolscheAlgebra.md
Autoren:	Sebastian Zug & André Dietrich & Fabian Bär & Copilot & Claude Code (Opus 5.5)



Lernziele: Nach dieser Vorlesung können Sie

- erklären, wie aus einer Spannung ein Logikpegel wird und warum Digitaltechnik robust ist,
- Boolesche Ausdrücke mit den Axiomen und Gesetzen – insbesondere De Morgan – umformen und vereinfachen,
- eine Schaltfunktion als Wahrheitstafel darstellen und aus der Tafel einen Ausdruck ablesen,
- Boolesche Funktionen als Bedingungen in C formulieren.

🗺️ **Diese Vorlesung in der Kurslandkarte →**

Rückblick: Hausaufgabe aus Vorlesung 01

Zuerst die Auflösung der Hausaufgabe aus der letzten Vorlesung. Beim Vergleich von Z1 und Z3 lohnt ein genauer Blick: Die Architektur ist dieselbe, nur die Bauelemente wechseln von Blechplatten zu Relais. Genau diese Trennung zwischen dem logischen Entwurf und seiner technischen Umsetzung zieht sich durch die heutige Vorlesung. Zuerst beschreiben wir mit der Booleschen Algebra, was eine Schaltung tun soll, danach betrachten wir, womit man sie baut.

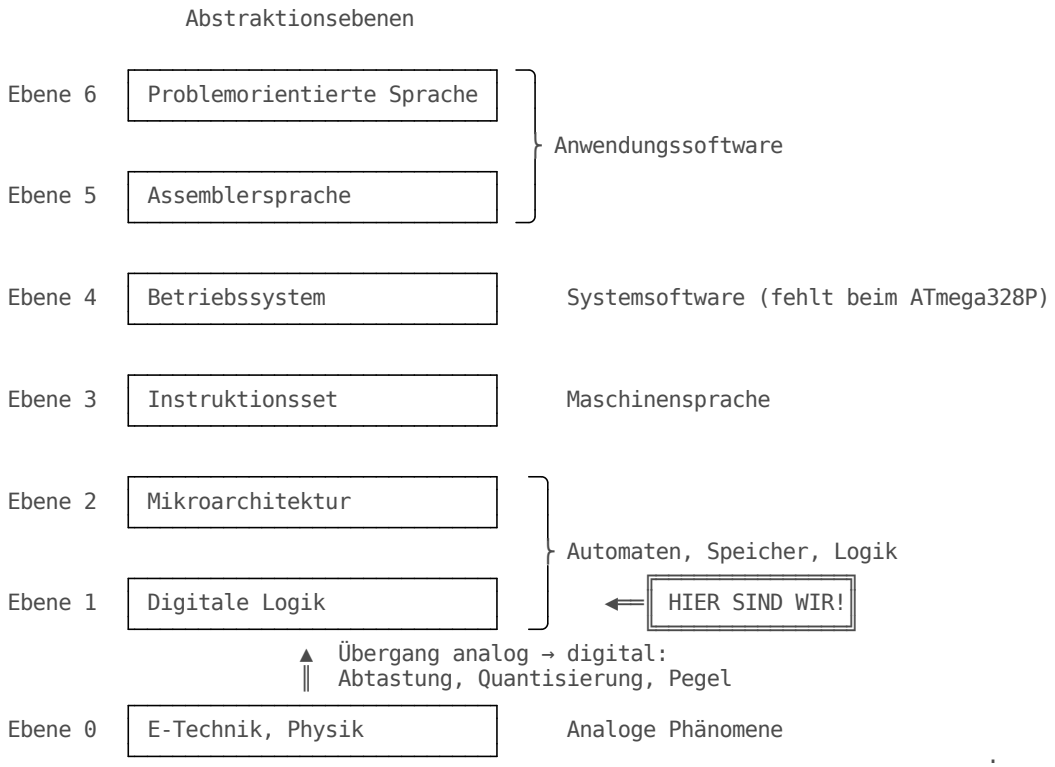
Z1 und Z3. Beide Rechner von Konrad Zuse arbeiten binär mit Gleitkommazahlen und lesen ihr Programm von einem gelochten Filmstreifen. Die Z1 (1938) ist rein mechanisch aufgebaut: Verschiebbare Blechplatten speichern und verknüpfen die Bits. Sie lief mit etwa 1 Hz und arbeitete nie zuverlässig, weil sich die Bleche verklemmten. Die Z3 (1941) setzt dieselbe Architektur mit rund 2000 Relais um – ein Relais ist ein elektrisch betätigter Schalter. Damit wurde sie schneller und vor allem zuverlässig.

„Bug“. Der Begriff für einen technischen Fehler ist älter als der Computer; Thomas Edison benutzte ihn schon in den 1870er-Jahren. Bekannt wurde eine Episode von 1947: Im Relaisrechner Harvard Mark II fand das Team um Grace Hopper eine Motte, die ein Relais blockierte, und klebte sie mit dem Vermerk „First actual case of bug being found“ ins Logbuch.

> Heute geht es genau um die Bausteine, aus denen die Z3 bestand: Schalter, die nur zwei Zustände kennen – und die Mathematik, mit der man sie beschreibt.

Digital vs. Analog

Im Schichtenmodell aus der ersten Vorlesung stehen wir jetzt auf Ebene eins, der digitalen Logik. Darunter liegt die Physik mit Spannungen und Strömen, die sich stetig ändern. Die Frage lautet also, wie aus einer stetigen physikalischen Größe die Nullen und Einsen werden, mit denen alle Ebenen darüber arbeiten.



Frage: Was bedeutet der Übergang von der Ebene der physikalischen Phänomene (0) auf die Ebene der digitalen Logik (1)?

Ein Digitalsignal entsteht in zwei Schritten, und beide verwerfen Information. Die Abtastung legt fest, wann gemessen wird, die Quantisierung, wie fein die Werte unterschieden werden. Im Diagramm sehen Sie das an der gestrichelten Treppe: Zwischen den Abtastzeitpunkten ist nichts bekannt, und aus sechs Komma zwei Volt und sechs Komma vier Volt wird beide Male sechs Volt. Drei Bit genügen für acht Stufen, denn zwei hoch drei ist acht.

Ein Analogsignal beschreibt den zeitlich kontinuierlichen Verlauf einer physikalischen Größe; es kann zu jedem Zeitpunkt jeden Wert annehmen:

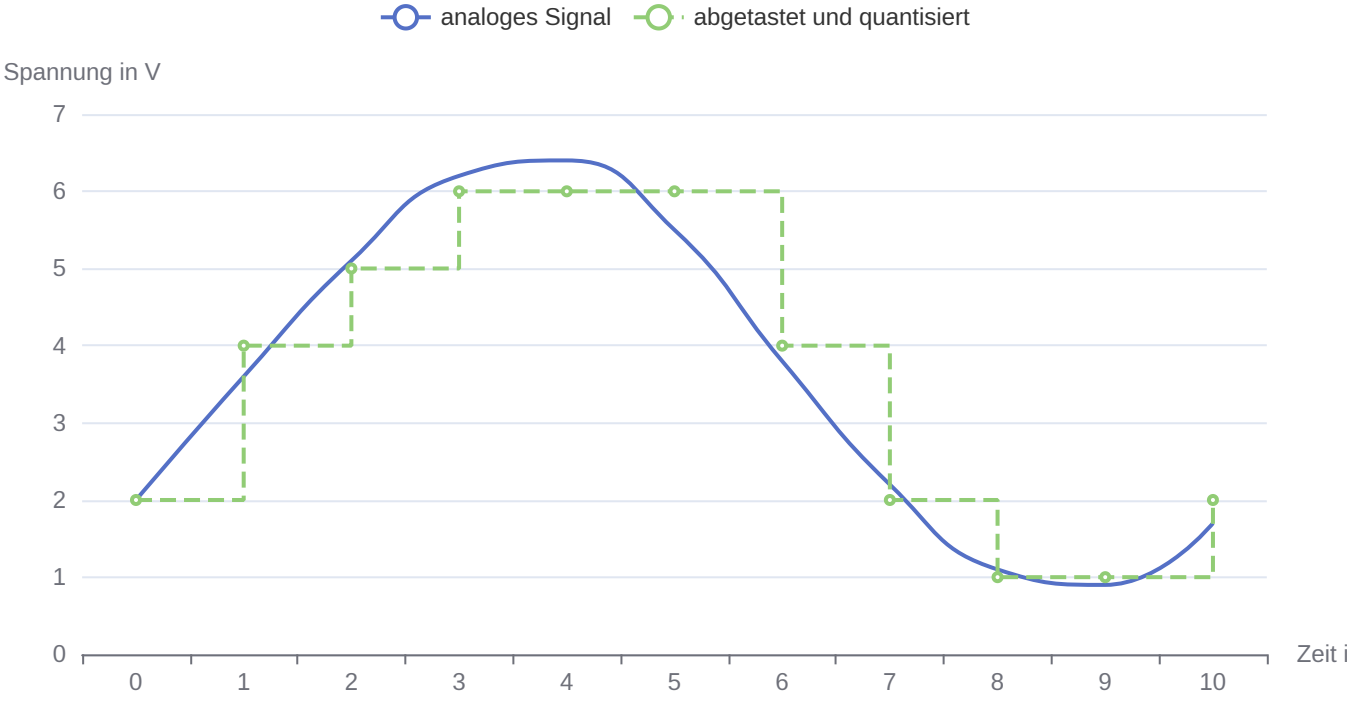
- Temperatur im Tagesverlauf
- Spannungswert am IC innerhalb der letzten nn Nanosekunden
- …

Ein Digitalsignal entsteht daraus in zwei Schritten:

- **Abtastung** – der Wert wird nur zu bestimmten Zeitpunkten erfasst (zeitdiskret),
- **Quantisierung** – jeder Messwert wird auf eine von endlich vielen Stufen gerundet (wertdiskret).

Die Stufen werden als Binärzahlen kodiert; die Auflösung der Quantisierung gibt man deshalb in Bit an. Im Diagramm wird ein Signal alle 1 ms abgetastet und auf ganze Volt gerundet, das entspricht einem Wandler mit 3 Bit (8 Stufen von 0 bis 7 V). Wie der Analog-Digital-Wandler im ATmega328P

das macht, sehen Sie in Vorlesung 13.



Die Abtastung und Bildung des Digitalsignals erfolgt üblicherweise in konstanten Zeitintervallen, allerdings ist dies nicht zwingend notwendig.

Exkurs: Kontrastprogramm - Analoge Rechner

> *Zum Selbststudium – in der Vorlesung nur kurz.*

Als Kontrast der Rechner aus Vorlesung 01, der gar nicht digitalisiert: Im Analogrechner ist eine Spannung selbst die Rechengröße, Verstärkerschaltungen addieren und integrieren sie. Die Tabelle zeigt den Preis dafür: Jede Temperaturänderung und jede Bauteiltoleranz verfälscht das Ergebnis unmittelbar. Ein Digitalsignal verträgt dagegen kleine Abweichungen, solange eindeutig bleibt, ob null oder eins gemeint ist. Ganz verschwunden ist das analoge Rechnen trotzdem nicht: Für KI-Beschleuniger wird es gerade wiederentdeckt, weil es so wenig Energie braucht. Wie groß diese Abweichungen sein dürfen, klären wir auf der nächsten Folie.

Im Analogrechner (Vorlesung 01) ist eine Spannung selbst die Rechengröße; Verstärkerschaltungen addieren, multiplizieren und integrieren sie ohne den Umweg über Ziffern.

	Analoge Hardware	Digitale Hardware
Vorteile	Multiplikation und Addition einfach realisierbar	unempfindlich gegen Störungen (Rauschen)
	vergleichsweise schnell	einfacher Entwurf
		beliebig hohe Präzision möglich
Nachteile	Temperaturabhängigkeit	vergleichsweise hoher Energieverbrauch
	nichtlineare Bauteile	
	Präzision durch Bauteiltoleranzen begrenzt	
	Langzeitspeicherung von Daten schwierig	

Digitaltechnik ermöglicht eine einfache Realisierung robuster Hardware.

Und heute? Als Universalrechner sind Analogrechner Geschichte, das analoge Rechnen kommt aber gerade zurück: KI-Beschleuniger berechnen die Multiplikation von Matrix und Vektor als Ströme in Speicherzellen – sehr sparsam, aber mit genau den Nachteilen aus der Tabelle (Rauschen, Drift, begrenzte Präzision). Für Lehre und Simulation von Differentialgleichungen gibt es wieder kleine Analogrechner, etwa den quelloffenen THE ANALOG THING der Firma anabrid.

Typische Fehlvorstellung

„Der ENIAC wurde mit Steckern programmiert, also war er ein Analogrechner.“ Das Stecken erinnert an den Analogrechner, aber der ENIAC rechnete digital – mit Ziffern, nicht mit Spannungen als Rechengröße.

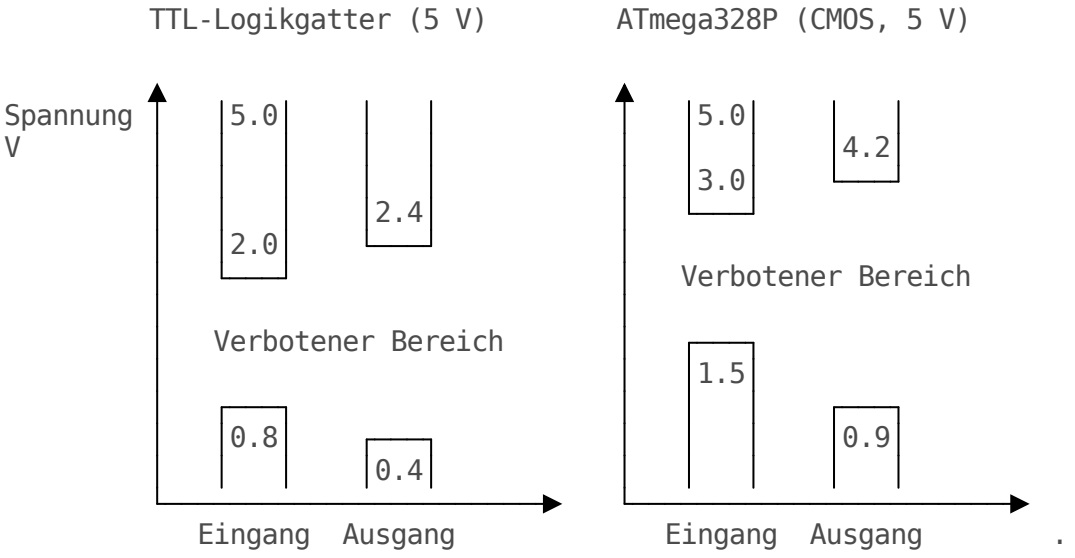
Pegel

Ein Logikpegel ist die Quantisierung von eben, nur mit zwei Stufen statt acht, also mit einem Bit. Ein digitaler Baustein muss jede Spannung eindeutig als null oder eins deuten. Deshalb legt jeder Standard zwei erlaubte Bereiche fest, Low unten und High oben, und dazwischen einen verbotenen Bereich, für den nichts garantiert wird. Lesen Sie das Diagramm spaltenweise: Für jeden Baustein steht links, was ein Eingang akzeptiert, und rechts, was ein Ausgang liefert. Vergleichen Sie beide Spalten, bevor Sie die Frage beantworten.

> Digitalisierung: Aufteilung des kontinuierlichen Spektrums in erlaubte und verbotene Bereiche

Ein Logikpegel ist der einfachste Fall der Quantisierung von eben: zwei Stufen statt acht, also **1 Bit** – mit einer Lücke dazwischen, damit kleine Abweichungen nicht schaden. Abgetastet wird dabei zunächst nicht; ein Gatter wertet die Spannung ständig aus. Wann ein Wert gilt, legt erst ein Takt fest, den wir mit den Flip-Flops einführen (Vorlesung 06 und 07).

Verschiedene Standards definieren unterschiedliche Spannungsbereiche für einen High- und einen Low-Pegel. Dazwischen befindet sich der verbotene Bereich. Links sehen Sie den klassischen TTL-Standard (was davon heute noch gilt, steht im Exkurs „Schaltertechnik und Pegel in der Praxis“), rechts die Werte des ATmega328P bei 5 V Versorgung aus dessen Datenblatt [ATmega].



> **Frage:** Warum ist der verbotene Bereich am Eingang schmaler als am Ausgang?

Entscheidend ist die Asymmetrie: Ein Ausgang verspricht mehr, als ein Eingang verlangt. Beim ATmega328P liefert ein Ausgang im High-Zustand selbst unter Last, bei zwanzig Milliampere, noch mindestens vier Komma zwei Volt, ein Eingang erkennt aber schon ab drei Volt sicher High. Die Lücke von eins Komma zwei Volt ist die Reserve für Spannungsabfälle auf Leitungen und für Störungen. Für Low gilt dasselbe: Der Ausgang liefert höchstens null Komma neun Volt, der Eingang akzeptiert bis eins Komma fünf Volt.

Ein Ausgang garantiert engere Grenzen, als ein Eingang verlangt. Bei TTL liefert ein Ausgang im High-Zustand mindestens 2,4 V, ein Eingang erkennt aber schon ab 2,0 V sicher High. Die Differenz von 0,4 V heißt **Störabstand**: So viel darf das Signal auf dem Weg durch Leitungen und Störungen verlieren, ohne falsch erkannt zu werden. Beim ATmega328P sind es im High-Bereich 4,2 V − 3,0 V = 1,2 V, im Low-Bereich 1,5 V − 0,9 V = 0,6 V. Die Ausgangswerte garantiert das Datenblatt bei 20 mA Laststrom (bis 85 °C); ohne Last liegt ein Ausgang fast bei 5 V bzw. 0 V.

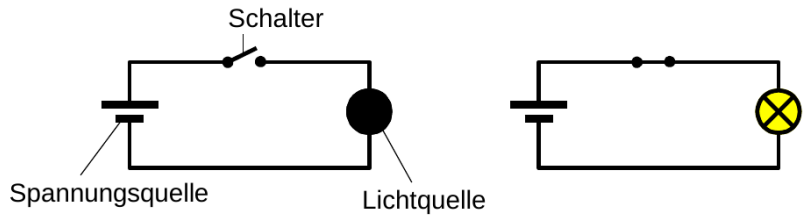
Ein Sensor liefert 2,0 V an einen digitalen Eingang des ATmega328P (5 V). Was liest `digitalRead()`?

- ◯ sicher `LOW`
- ◯ sicher `HIGH`
- ◯ nicht vorhersagbar – die Spannung liegt im verbotenen Bereich

Motivation

Bevor wir rechnen, ein Blick auf den einfachsten digitalen Baustein, den Schalter. Er kennt nur zwei Zustände, offen und geschlossen, und die Lampe folgt ihm. Was in den ersten Millisekunden nach dem Umschalten passiert, etwa das Prellen der Kontakte, blenden wir bewusst aus; uns interessiert nur der Zustand danach.

Nehmen wir folgende einfache Schaltung an:



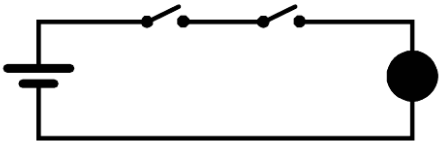
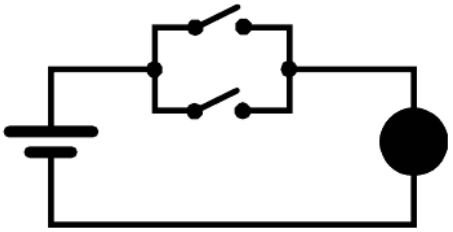
Wir betrachten den Schalter mit seinen 2 Zuständen als Input und die Glühlampe als Output. Das Übergangsverhalten wird ignoriert.

Input	Output
Schalter geschlossen	Lampe leuchtet
Schalter offen	Lampe leuchtet nicht

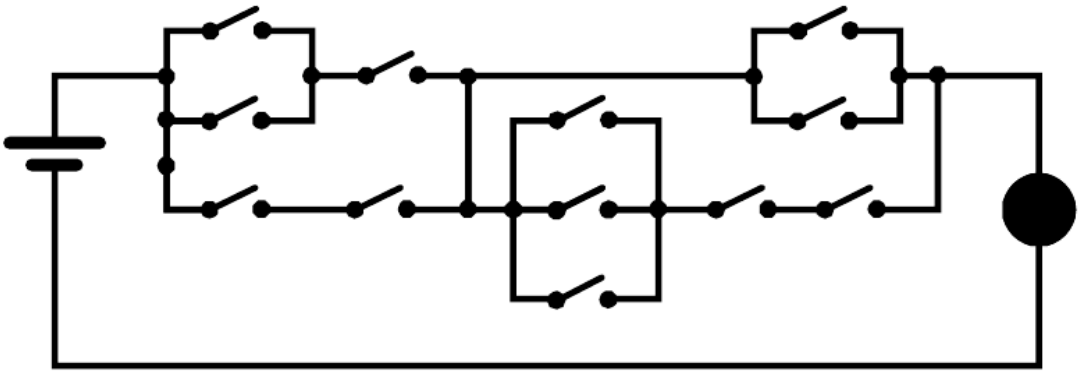
Mit zwei Schaltern entstehen die beiden Grundmuster, die Sie sprachlich schon kennen. In Reihe muss der Strom durch beide Schalter, die Lampe leuchtet also nur, wenn der erste und der zweite geschlossen sind. Parallel genügt ein geschlossener Weg, also der erste oder der zweite. Damit

werden Und und Oder physikalisch greifbar. Dieselben Muster haben Sie gestern bei den Relais gesehen; ersetzt man die Handschalter durch Relaiskontakte, hat man die Bausteine der Z3.

Für zwei Schalter (Inputs) lassen sich darauf aufbauend zwei grundlegende Schaltungsmuster entwerfen:

Reihenschaltung	**Parallelschaltung**
	
Die Lampe leuchtet, wenn der erste und der zweite Schalter geschlossen werden	Die Lampe leuchtet, wenn der erste oder der zweite Schalter (oder beide) geschlossen werden

Bei zwölf Schaltern hilft Hinschauen allein nicht mehr. Zählen Sie zuerst die Wege mit genau drei geschlossenen Schaltern und dann alle möglichen Stellungen; jeder weitere Schalter verdoppelt die Zahl der Kombinationen. Das Ergebnis zeigt, warum wir für Schaltungen Rechenregeln brauchen statt einer Liste aller Fälle.



Mit genau drei geschlossenen Schaltern kann die Lampe leuchten. Auf wie viele Arten?

Wie viele Schalterstellungen gibt es insgesamt?

Die Abstraktion ersetzt die Physik durch zwei Symbole, null und eins. Welche Spannung man eins nennt, ist eine Vereinbarung: Bei positiver Logik ist die hohe Spannung die Eins, bei negativer Logik die niedrige. Der Taster mit Pull-up-Widerstand ist ein solcher Fall, denn gedrückt liegt sein Eingang auf Low.

> Wir brauchen eine Abstraktion, um die Abbildung von digitalen Eingängen E auf einen digitalen Ausgang A repräsentieren und analysieren zu können.

Dazu beschreiben wir die Wirkung des elektrischen Stromes

- Stromfluss / kein (oder ein sehr geringer) Stromfluss
- Spannung / keine (oder eine sehr geringe) Spannung

… aus Sicht der Logik anhand von Zuständen

- an / aus
- wahr / falsch
- 1 / 0

Welcher Spannung man die 1 zuordnet, ist eine Festlegung. Üblich ist **positive Logik**: hohe Spannung = 1. Bei **negativer Logik** steht die niedrige Spannung für die 1. Häufig ist auch nur ein einzelnes Signal bei Low aktiv, man nennt es dann *active low*. Ein Beispiel kennen Sie aus Vorlesung 00: Ein **Pull-up-Widerstand** verbindet den Eingang mit der Versorgungsspannung und hält ihn auf High, solange der Taster offen ist. Der gedrückte Taster verbindet den Eingang mit GND, der Pin liest LOW. Der ATmega328P hat diesen Widerstand eingebaut, `pinMode(pin, INPUT_PULLUP)` schaltet ihn ein.

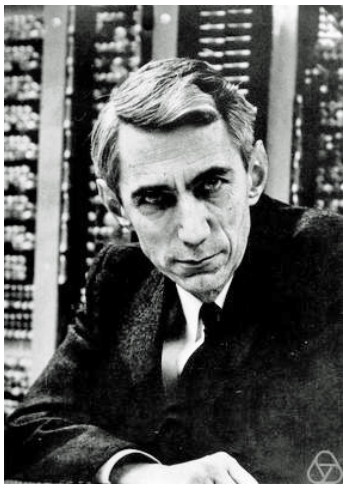
> Wie aber können logische Grundverknüpfungen identifiziert werden? Auf welchem Wege lassen diese sich praktisch realisieren?

Axiome und Gesetze

Gestern haben Sie George Boole kennengelernt, der 1854 das logische Schließen als Algebra formulierte, und am Ende den Ausblick auf Claude Shannon. Shannon machte 1937 den Schritt zur Technik: Er zeigte, dass Schaltungen aus Relaiskontakten genau diesen Regeln gehorchen – Reihen- und Parallelschaltung, wie eben bei den Schaltern, entsprechen den Verknüpfungen der Algebra. Seitdem kann man Schaltungen auf dem Papier vergleichen, umformen und vereinfachen, bevor man ein einziges Bauteil verdrahtet.

Historische Entwicklung:

- Aristoteles (384-322 v.Chr.) begründet die „Syllogistik“, die Lehre von den logischen Schlussformen; die Stoiker entwickeln daneben eine Aussagenlogik.
- George Boole (1815-1864) formalisiert 1854 das Schließen als Algebra in *„An Investigation of the Laws of Thought“* (Vorlesung 01).
- Claude Shannon (1916-2001) zeigt in seiner Masterarbeit *„A Symbolic Analysis of Relay and Switching Circuits“* (MIT 1937, veröffentlicht 1938), dass man Relaisschaltungen mit der Booleschen Algebra beschreiben und vereinfachen kann. Damit wird aus Logik Schaltungstechnik.



Claude Shannon Jacobs, Konrad, Shannon, Claude, https://commons.wikimedia.org/wiki/File:ClaudeShannon_MFO3807.jpg

> Problem: Gibt es ein Verfahren:
>
> - um die Äquivalenz zweier Schaltungen formal nachzuweisen ?
> - um Schaltungen auf einfache Weise zu transformieren ?
> - um minimale Schaltungen zu entwerfen ?

Lösung: Boolesche Algebra basierend auf den Vorarbeiten von G. Boole aus dem Jahre 1854

- zwei Werte: 0 und 1
- drei Operationen: die zweistelligen Verknüpfungen $+$ + und $\cdot$ · sowie die einstellige Negation $\bar{a}$a
- vier Axiome

Für dieselben drei Operationen gibt es mehrere Schreibweisen. In dieser Vorlesung steht das Pluszeichen für Oder, der Malpunkt für Und und ein Strich über der Variablen für Nicht. Das Plus ist dabei kein Rechenplus, denn eins oder eins ergibt eins. In C haben dieselben Verknüpfungen eigene Operatoren, dazu mehr am Ende der Vorlesung.

Bedeutung	Schreibweise hier	Aussagenlogik	C	Python
ODER	$a + b$a + b	$a \vee b$a ∨ b	`a || b`	`a or b`
UND	$a \cdot b$a · b oder abab	$a \wedge b$a ∧ b	`a && b`	`a and b`
NICHT	$\bar{a}$a	$\neg a$¬a	`!a`	`not a`

Achtung: $+$ + ist hier kein Rechenplus. Es gilt $1 + 1 = 1$1 + 1 = 1. Wie in der Schulmathematik bindet $\cdot$· stärker als $+$ +, und der Punkt darf entfallen: $ab + c = (a \cdot b) + c$ab + c = (a · b) + c. Ein Negationsstrich gilt für alles, was unter ihm steht: $\bar{a + b}$a + b negiert die ganze Summe, $\bar{a} + b$ a + b nur $\bar{a}$a.

Nach Huntington ist eine Boolesche Algebra eine Menge BB mit zwei Verknüpfungen $+$ + und $\cdot$·, die für alle $a, b, c \in B$a, b, c ∈ B die folgenden vier Axiome erfüllt. Ein Axiom ist eine Grundregel, die man nicht beweist; alles Weitere folgt daraus. Für Schaltungen genügt $B = \{0, 1\}$B = {0, 1}, dort lassen sich alle Axiome an der Wertetabelle nachprüfen.

Die vier Axiome treten jeweils als Paar auf, eine Zeile für Oder und eine für Und. Kommutativität erlaubt das Vertauschen, Distributivität das Ausmultiplizieren und Ausklammern, und zwar anders als in der Schulmathematik auch für Oder über Und. Null ist das neutrale Element für Oder, Eins das für Und. Das Komplement-Axiom schließlich sagt: a oder nicht a ergibt eins, a und nicht a ergibt null.

Axiom	Definition
Kommutativität	$a + b = b + a$ $a \cdot b = b \cdot a$ a + ba · b = b + a = b · a
Distributivität	$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$ $a + (b \cdot c) = (a + b) \cdot (a + c)$ a · (b + c)a + (b · c) = (a · b) + (a · c) = (a + b) · (a + c)
Existenz neutraler Elemente	$0 + a = a$ $1 \cdot a = a$ 0 + a1 · a = a = a
Existenz von Komplementen	zu jedem aa gibt es $\bar{a}$a mit $a + \bar{a} = 1$ $a \cdot \bar{a} = 0$ a + aa · a = 1 = 0

Aus den vier Axiomen folgen alle weiteren Gesetze, man muss sie nicht zusätzlich annehmen. Besonders oft brauchen Sie später Idempotenz, Absorption und die De Morgan'sche Regel. De Morgan liest sich so: nicht, in Klammern a oder b, ist gleich nicht a und nicht b. Ein Nicht über einem ganzen Ausdruck negiert also jede Variable einzeln und macht aus Oder ein Und und umgekehrt.

Aus dieser Definition lassen sich die zugehörigen Gesetze der booleschen Algebra ableiten:

Gesetz	Definition
Dominanz (Null- und Einselement)	$1 + a = 1$ $0 \cdot a = 0$ 1 + a0 · a = 1 = 0
Assoziativität	$a + (b + c) = (a + b) + c = a + b + c$ $a \cdot (b \cdot c) = (a \cdot b) \cdot c = a \cdot b \cdot c$ a + (b + c)a · (b · c) = (a + b) + c = a + b + c = (a · b) · c = a · b · c
Idempotenzgesetze	$a + a = a$ $a \cdot a = a$ a + aa · a = a = a
Absorptionsgesetz	$a + (a \cdot b) = a$ $a \cdot (a + b) = a$ a + (a · b)a · (a + b) = a = a
Doppelnegation	$a = \bar{\bar{a}}$ a = a
De Morgan'sche Regel	$a \bar{+} b = \bar{a} \cdot \bar{b}$ $a \bar{\cdot} b = \bar{a} + \bar{b}$ a + ba · b = a · b = a + b

De Morgan in Worten: Ein Strich über dem ganzen Ausdruck wandert auf jede einzelne Variable; dabei wird aus $+$ + ein $\cdot$ · und aus $\cdot$ · ein $+$ +.

Kurz geprüft: Welcher Ausdruck ist gleichwertig zu $a \bar{\cdot} b$a · b – in C `!(a && b)`?

- ◯ $\bar{a} \cdot \bar{b}$a · b, in C `!a && !b`
- ◯ $\bar{a} + \bar{b}$a + b, in C `!a || !b`
- ◯ $a + b$a + b, in C `a || b`

Lesen Sie die Herleitung Zeile für Zeile; rechts steht jeweils das Axiom, das den Schritt erlaubt. Zuerst wird a oder a mit eins verknüpft, das erlaubt das neutrale Element, und dann wird diese Eins durch a oder nicht a ersetzt, das Komplement-Axiom rückwärts gelesen. Das Distributivgesetz fasst den Ausdruck zu a oder, in Klammern, a und nicht a zusammen. Die Klammer ist nach dem Komplement null, und a oder null ergibt a.

Wie so eine Ableitung aussieht, zeigt das Idempotenzgesetz – jeder Schritt stützt sich auf ein Axiom:

$$
\begin{aligned}
a + a &= (a + a) \cdot 1 && \text{(neutrales Element: } 1 \cdot x = x\text{)} \\
&= (a + a)(a + \bar{a}) && \text{(Komplement: } x + \bar{x} = 1\text{)} \\
&= a + a\bar{a} && \text{(Distributivität: } (x + y)(x + z) = x + yz\text{)} \\
&= a + 0 && \text{(Komplement: } x\bar{x} = 0\text{)} \\
&= a && \text{(neutrales Element: } 0 + x = x\text{)}
\end{aligned}
$$

a + a = (a + a) · 1 = (a + a)(a + a) = a + aa = a + 0 = a(neutrales Element: 1 · x = x)

(Komplement: x + x = 1)(Distributivita¨t: (x + y)(x + z) = x + yz)(Komplement: xx = 0)
(neutrales Element: 0 + x = x)

Die Axiome für das neutrale Element sind mit der 1 bzw. 0 links notiert; dass sie auch rechts stehen darf, liefert die Kommutativität.

Für Vorerfahrene: Leiten Sie ebenso $1 + a = 1$1 + a = 1 und die De Morgan'sche Regel aus den vier Axiomen her. Hinweis für De Morgan: Zeigen Sie, dass $\bar{a}\bar{b}$a b die beiden Komplement-Axiome für $a + b$a + b erfüllt, und begründen Sie, warum das Komplement eindeutig ist.

Lösungsskizze ›

Das Dualitätsprinzip halbiert die Arbeit, denn die Gesetze kommen paarweise. Vertauschen Sie in einer gültigen Gleichung Und mit Oder sowie Null mit Eins, so erhalten Sie wieder eine gültige Gleichung. Die Tabelle zeigt solche Paare zeilenweise, etwa a oder nicht a ergibt eins und dual dazu a und nicht a ergibt null. Das funktioniert, weil schon die vier Axiome paarweise dual sind; jeder Beweis lässt sich deshalb Schritt für Schritt spiegeln. Shannon selbst hat 1938 übrigens genau dual gerechnet: Er gab einem geschlossenen Kontakt den Wert null, weil er die Hemmung des Stromflusses zählte. Bei ihm war die Reihenschaltung deshalb ein Plus und die Parallelschaltung ein Mal – und alle Gesetze galten trotzdem.

Merke

Dualitätsprinzip: Zu jeder gültigen Gleichung der Booleschen Algebra gehört eine duale Gleichung, die ebenfalls gilt.

Ersetzt man gleichzeitig in einem Axiom UND durch ODER und ODER durch UND sowie 1 durch 0 und 0 durch 1, so erhält man das zu diesem Axiom gehörige duale Axiom. Führt man diese Ersetzung in einem Theorem aus, so erhält man das zu diesem Theorem gehörige duale Theorem. Variablen und Negationsstriche bleiben dabei unverändert.

Gesetz	Form 1	Form 2
neutrales Element	$0 + a = a$0 + a = a	$1 \cdot a = a$1 · a = a
Dominanz	$1 + a = 1$1 + a = 1	$0 \cdot a = 0$0 · a = 0
Idempotenz	$a + a = a$a + a = a	$a \cdot a = a$a · a = a
Komplement	$a + \bar{a} = 1$a + a = 1	$a \cdot \bar{a} = 0$a · a = 0
Distributivität	$a + (b \cdot c) = (a + b) \cdot (a + c)$a + (b · c) = (a + b) · (a + c)	$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$ a · (b + c) = (a · b) + (a · c)
De Morgan	$\overline{a + b} = \bar{a} \cdot \bar{b}$a + b = a · b	$\overline{a \cdot b} = \bar{a} + \bar{b}$a · b = a + b

Anwendung

Wozu das alles? Jeder Term einer Funktion wird später zu Gattern, und jedes eingesparte Gatter spart Chipfläche, Laufzeit und Energie. Im Beispiel wird zuerst a ausgeklammert; b oder nicht b ergibt eins, und a oder a und c ist nach dem Absorptionsgesetz einfach a. Die Variablen b und c haben auf das Ergebnis also gar keinen Einfluss.

Ziel der Umformung: Minimierung für die technische Realisierung

> **Warum Minimierung?** Jeder Term wird später zu Gattern auf dem Chip. Weniger Terme und Variablen bedeuten weniger Gatter, weniger Fläche, kürzere Laufzeiten und geringeren Energieverbrauch. Auch der ATmega328P besteht intern aus solchen Gattern – sein Rechenwerk und sein Befehlsdekoder sind Schaltfunktionen (Vorlesung 10 und 11).

Grundlegende Strategien zur Minimierung: Nutze die Booleschen Gesetze zur schrittweisen Reduktion der Terme.

Beispiel:

$a \cdot b + a \cdot \bar{b} + a \cdot c = a \cdot (b + \bar{b}) + a \cdot c = a \cdot 1 + a \cdot c = a + a \cdot c = a$
a · b + a · b + a · c = a · (b + b) + a · c = a · 1 + a · c = a + a · c = a (Absorption!)

Aus drei Produkttermen wird eine einzige Variable.

Im ersten Beispiel steckt ein Kunstgriff. Der mittlere Term passt zu beiden Nachbarn: Mit dem ersten Term teilt er x eins und x zwei, mit dem dritten x eins und x drei. Weil a oder a wieder a ergibt, darf man ihn per Idempotenz verdoppeln, ohne die Funktion zu ändern. Eine Kopie verschmilzt dann mit dem ersten Term, die andere mit dem dritten, und es bleibt x eins x zwei oder x eins x drei.

Anwendungsbeispiel 1

$$
\begin{aligned}
f(x_1, x_2, x_3) &= x_1 \cdot x_2 \cdot \bar{x_3} + x_1 \cdot x_2 \cdot x_3 + x_1 \cdot \bar{x_2} \cdot x_3 \\
&= x_1 \cdot x_2 \cdot \bar{x_3} + x_1 \cdot x_2 \cdot x_3 + x_1 \cdot x_2 \cdot x_3 + x_1 \cdot \bar{x_2} \cdot x_3 && \text{(Idempotenz)} \\
&= x_1 \cdot x_2 \cdot (\bar{x_3} + x_3) + x_1 \cdot x_2 \cdot x_3 + x_1 \cdot \bar{x_2} \cdot x_3 && \text{(Assoziativität, Distributivität)} \\
&= x_1 \cdot x_2 \cdot (\bar{x_3} + x_3) + x_1 \cdot x_3 \cdot x_2 + x_1 \cdot x_3 \cdot \bar{x_2} && \text{(Kommutativität)} \\
&= x_1 \cdot x_2 \cdot (\bar{x_3} + x_3) + x_1 \cdot x_3 \cdot (x_2 + \bar{x_2}) && \text{(Distributivität)} \\
&= x_1 \cdot x_2 \cdot (1) + x_1 \cdot x_3 \cdot (1) && \text{(Komplement)} \\
&= x_1 \cdot x_2 + x_1 \cdot x_3 && \text{(neutrales Element)}
\end{aligned}
$$

f(x1, x2, x3) = x1 · x2 · x3 + x1 · x2 · x3 + x1 · x2 · x3 = x1 · x2 · x3 + x1 · x2 · x3 + x1 · x2 · x3 + x1 · x2

· x3 = x1 · x2 · (x3 + x3) + x1 · x2 · x3 + x1 · x2 · x3 = x1 · x2 · (x3 + x3) + x1 · x3 · x2 + x1 · x3 · x2 = x1 · x2 · (x3 + x3) + x1 · x3 · (x2 + x2) = x1 · x2 · (1) + x1 · x3 · (1) = x1 · x2 + x1 · x3(Idempotenz) (Assoziativita¨t, Distributivita¨t)(Kommutativita¨t)(Distributivita¨t)(Komplement)(neutrales Element)

Bilanz: Vorher drei UND-Gatter mit je drei Eingängen und ein ODER-Gatter mit drei Eingängen. Dazu kommen zwei Inverter für $\bar{x_2}$x2 und $\bar{x_3}$x3. Nachher zwei UND-Gatter und ein ODER-Gatter mit je zwei Eingängen und kein Inverter. Ausgeklammert, $x_1(x_2 + x_3)$x1(x2 + x3), genügen ein ODER- und ein UND-Gatter.

Das zweite Beispiel hat vier Variablen und fünf Terme. Suchen Sie zuerst Paare von Termen, die sich nur in einer einzigen Variablen unterscheiden, einmal negiert und einmal nicht. Solche Paare lassen sich zusammenfassen, und die betreffende Variable fällt heraus.

Anwendungsbeispiel 2

$$f(w, x, y, z) = \bar{w}x\bar{y}\bar{z} + \bar{w}x\bar{y}z + w\bar{x}y\bar{z} + wx\bar{y}\bar{z} + wx\bar{y}z$$

f(w, x, y, z) = wxy z + wxyz + wxyz + wxy z + wxyz

Die Umformung fasst zweimal Paare zusammen, die sich nur in z unterscheiden, und danach die beiden Ergebnisse, die sich nur in w unterscheiden. Jedes Mal läuft dasselbe Muster ab: ausklammern, das Komplement ergibt eins, das neutrale Element lässt die Eins verschwinden. Der Term w, nicht x, y, nicht z hat keinen Partner und bleibt stehen. In der nächsten Vorlesung macht das KV-Diagramm solche Paare auf einen Blick sichtbar.

$$
\begin{aligned}
f(w,x,y,z) &= \bar{w}x\bar{y}\bar{z} + \bar{w}x\bar{y}z + w\bar{x}y\bar{z} + wx\bar{y}\bar{z} + wx\bar{y}z \\
&= \bar{w}x\bar{y}\,(\,\bar{z} + z\,) + wx\bar{y}\,(\,\bar{z} + z\,) + w\bar{x}y\bar{z} && (\text{Kommutativität, } 2\times \text{Distributivität}) \\
&= \bar{w}x\bar{y}\cdot 1 + wx\bar{y}\cdot 1 + w\bar{x}y\bar{z} && (\text{Komplement}) \\
&= \bar{w}x\bar{y} + wx\bar{y} + w\bar{x}y\bar{z} && (\text{neutrales Element}) \\
&= x\bar{y}\bar{w} + x\bar{y}w + w\bar{x}y\bar{z} && (2\times \text{Kommutativität}) \\
&= x\bar{y}\,(\,\bar{w} + w\,) + w\bar{x}y\bar{z} && (\text{Distributivität}) \\
&= x\bar{y}\cdot 1 + w\bar{x}y\bar{z} && (\text{Komplement}) \\
&= x\bar{y} + w\bar{x}y\bar{z} && (\text{neutrales Element})
\end{aligned}
$$

f(w, x, y, z) = wxy z + wxyz + wxyz + wxy z + wxyz = wxy (z + z) + wxy (z + z) + wxyz = wxy · 1 + wxy · 1 + wxyz = wxy + wxy + wxyz = xy w + xyw + wxyz = xy (w + w) + wxyz = xy · 1 + wxyz = xy + wxyz (Kommutativita¨t, 2 × Distributivita¨t)(Komplement)(neutrales Element)(2 × Kommutativita¨t) (Distributivita¨t)(Komplement)(neutrales Element)

> **Bilanz:** Vorher fünf Terme mit je vier Variablen, also fünf UND-Gatter mit vier Eingängen und ein ODER-Gatter mit fünf Eingängen. Nachher ein UND-Gatter mit zwei und eines mit vier Eingängen sowie ein ODER-Gatter mit zwei Eingängen.

Im dritten Beispiel formen Sie selbst um. Gehen Sie von innen nach außen vor: Was ergibt die Klammer x eins oder nicht x eins, und was bewirken zwei Negationsstriche übereinander? Für den zweiten Term brauchen Sie die De Morgan'sche Regel.

Anwendungsbeispiel 3

$$
f(x_1,x_2) = \bar{x}_1x_2(\bar{\bar{x}_1} + \bar{x}_1) + x_1x_2\bar{x}_1
$$

f(x1, x2) = x1x2(x1 + x1) + x1x2x1

> Jetzt sind Sie dran! Die beiden äußeren Striche des ersten Terms reichen über den ganzen Ausdruck $\bar{x}_1x_2(x_1 + \bar{x}_1)$x1x2(x1 + x1). Welche Funktion ergibt sich?

- $f = x_2$f = x2
- $f = x_1x_2 + \bar{x}_1\bar{x}_2$f = x1x2 + x1x2
- $f = \bar{x}_1x_2 + x_1\bar{x}_2$f = x1x2 + x1x2, das exklusive ODER (XOR)
- $f = 1$f = 1

Die Lösung arbeitet den ersten Term von innen nach außen ab: Komplement, neutrales Element, Doppelnegation. Im zweiten Term löst De Morgan die Negation über x zwei und x eins auf, danach wird ausmultipliziert. Der entstehende Teilterm x eins und nicht x eins ist nach dem Komplement-Axiom null und fällt weg. Übrig bleiben genau die beiden Fälle, in denen x eins und x zwei verschieden sind, also das exklusive Oder.

$$
\begin{aligned}
f(x_1, x_2) &= \bar{x}_1x_2(\bar{\bar{x}_1} + \bar{x}_1) + x_1x_2\bar{x}_1 \\
&= \bar{x}_1\bar{\bar{x}_2} \cdot 1 + x_1x_2\bar{x}_1 && (\text{Komplement: } x_1 + \bar{x}_1 = 1) \\
&= \bar{x}_1\bar{\bar{x}_2} + x_1x_2\bar{x}_1 && (\text{Neutrales Element: } a \cdot 1 = a) \\
&= \bar{x}_1x_2 + x_1x_2\bar{x}_1 && (\text{Doppelnegation: } \bar{\bar{a}} = a) \\
&= \bar{x}_1x_2 + x_1(\bar{x}_2 + \bar{x}_1) && (\text{De Morgan: } \overline{a \cdot b} = \bar{a} + \bar{b}) \\
&= \bar{x}_1x_2 + x_1\bar{x}_2 + x_1\bar{x}_1 && (\text{Distributivität}) \\
&= \bar{x}_1x_2 + x_1\bar{x}_2 + 0 && (\text{Komplement: } x_1\bar{x}_1 = 0) \\
&= \bar{x}_1x_2 + x_1\bar{x}_2 && (\text{Neutrales Element: } a + 0 = a) \\
&= x_1 \oplus x_2 && (\text{Definition XOR: } a \oplus b = \bar{a}b + a\bar{b})
\end{aligned}
$$

f(x1, x2) = x1x2(x1 + x1) + x1x2x1 = x1x2 · 1 + x1x2x1 = x1x2 + x1x2x1 = x1x2 + x1x2x1 = x1x2 + x1

(x2 + x1) = x1x2 + x1x2 + x1x1 = x1x2 + x1x2 + 0 = x1x2 + x1x2 = x1 ⊕ x2(Komplement: x1 + x1 = 1)
(Neutrales Element: a · 1 = a)(Doppelnegation: a = a)(De Morgan: a · b = a + b)(Distributivita¨t)
(Komplement: x1x1 = 0)(Neutrales Element: a + 0 = a)(Definition XOR: a ⊕ b = ab + ab)

Ein unübersichtlicher Ausdruck beschreibt hier am Ende eine einzige Grundfunktion. Ohne Umformung hätte man alle Negationen, Und- und Oder-Gatter einzeln verdrahtet. Ob sich ein Ausdruck vereinfachen lässt, sieht man ihm von außen selten an; deshalb lohnt das systematische Umformen.

Bilanz: Aus einem Ausdruck mit fünf Negationen wird ein einziges XOR-Gatter.

Schaltfunktionen

Jetzt wechseln wir den Blickwinkel: Statt einzelner Ausdrücke betrachten wir Funktionen, die Bitmuster auf Bitmuster abbilden. Eine Funktion mit mehreren Ausgängen zerlegt man in mehrere Funktionen mit je einem Ausgang. Die Zahl zwei hoch zwei hoch n ergibt sich so: Die Wahrheitstafel hat zwei hoch n Zeilen, und jede Zeile kann unabhängig null oder eins sein. Ausdrücke und Schaltnetze gibt es für dieselbe Funktion viele, die Wahrheitstafel dagegen ist eindeutig.

- Funktionen $f\colon \{0,1\}^n \to \{0,1\}^m$f : {0, 1}n → {0, 1}m mit $n, m \geq 1$n, m ≥ 1 werden auch als Schaltfunktionen bezeichnet
- Für $m = 1$m = 1, also $f\colon \{0,1\}^n \to \{0,1\}$f : {0, 1}n → {0, 1}, heißt die Schaltfunktion eine nn-stellige Boolesche Funktion – in Worten: nn Eingänge, ein Ausgang, alles nur 0 oder 1
- Jede Schaltfunktion $f\colon \{0,1\}^n \to \{0,1\}^m$f : {0, 1}n → {0, 1}m kann durch mm Boolesche Funktionen ausgedrückt werden
- Jede Boolesche Funktion lässt sich beschreiben
  - durch eine Wahrheitstabelle / Wahrheitstafel
  - durch einen booleschen Ausdruck (gebildet durch Boolesche Variablen und Operationen aus der Booleschen Algebra)
  - durch ein Schaltnetz aus logischen Gattern (ein Schaltnetz hat keinen Speicher; Schaltungen mit Speicher, sogenannte Schaltwerke, folgen ab Vorlesung 06)
- Es gibt 2^{2^n}22n verschiedene nn-stellige Boolesche Funktionen (also 16 zweistellige, 256 dreistellige, 65536 vierstellige, …): Die Wahrheitstafel hat 2^n2n Zeilen, und in jeder Zeile kann ff den Wert 0 oder 1 haben.

> **Merke**
>
> Die Wahrheitstafel (bei fester Reihenfolge der Variablen) beschreibt eine Boolesche Funktion eindeutig. Zu jeder Funktion gibt es dagegen viele gleichwertige Ausdrücke und Schaltnetze; eindeutig ist unter ihnen nur eine Normalform wie die KDNF (Vorlesung 03).

Ein Und-Term ist nur dann eins, wenn jeder seiner Faktoren eins ist. Eine Zeile der Tafel erhält also den Wert eins, sobald mindestens einer der fünf Terme für diese Belegung eins ergibt. Zählen Sie die Belegungen binär von null null null bis eins eins eins hoch, dann vergessen Sie keine.

Stellen Sie eine Wahrheitstafel für folgende Schaltfunktion auf:

$$f(x_1, x_2, x_3) = \begin{aligned} &\bar{x}_1 \cdot \bar{x}_2 \cdot \bar{x}_3 + \\ &\bar{x}_1 \cdot \bar{x}_2 \cdot x_3 + \\ &x_1 \cdot \bar{x}_2 \cdot x_3 + \\ &x_1 \cdot x_2 \cdot \bar{x}_3 + \\ &x_1 \cdot \bar{x}_2 \cdot \bar{x}_3 \end{aligned}$$

f(x1, x2, x3) =x1 · x2 · x3+x1 · x2 · x3+x1 · x2 · x3+x1 · x2 · x3+x1 · x2 · x3

Bei drei Eingängen hat die Tafel $2^3 = 8$23 = 8 Zeilen. Damit keine fehlt, zählen Sie die Eingänge binär von 000 bis 111 hoch. Tragen Sie für jede Zeile den Wert von ff ein:

0	1	
◯	◯	$x_1x_2x_3 = 000$x1x2x3 = 000
◯	◯	$x_1x_2x_3 = 001$x1x2x3 = 001
◯	◯	$x_1x_2x_3 = 010$x1x2x3 = 010
◯	◯	$x_1x_2x_3 = 011$x1x2x3 = 011
◯	◯	$x_1x_2x_3 = 100$x1x2x3 = 100
◯	◯	$x_1x_2x_3 = 101$x1x2x3 = 101
◯	◯	$x_1x_2x_3 = 110$x1x2x3 = 110
◯	◯	$x_1x_2x_3 = 111$x1x2x3 = 111

Die Tafel zeigt den Zusammenhang in beiden Richtungen. Jeder Term der Funktion trifft genau eine Zeile, nämlich die, in der seine nicht negierten Variablen eins und seine negierten Variablen null sind. Umgekehrt gewinnt man aus jeder Tafel eine Funktion, indem man für jede Zeile mit eins den passenden Minterm bildet und alle mit Oder verknüpft. So lässt sich jede Wahrheitstafel in einen Ausdruck übersetzen, auch wenn dieser selten der kürzeste ist.

x_1x1	x_2x2	x_3x3	f	Term
0	0	0	1	$\bar{x}_1 \cdot \bar{x}_2 \cdot \bar{x}_3$x1 · x2 · x3
0	0	1	1	$\bar{x}_1 \cdot \bar{x}_2 \cdot x_3$x1 · x2 · x3
0	1	0	0	
0	1	1	0	
1	0	0	1	$x_1 \cdot \bar{x}_2 \cdot \bar{x}_3$x1 · x2 · x3
1	0	1	1	$x_1 \cdot \bar{x}_2 \cdot x_3$x1 · x2 · x3
1	1	0	1	$x_1 \cdot x_2 \cdot \bar{x}_3$x1 · x2 · x3
1	1	1	0	

Jeder Term der Funktion ist ein Produkt, in dem **jede** Variable genau einmal vorkommt – negiert oder nicht. Ein solcher Term heißt **Minterm**; er ist genau in einer Zeile der Tafel 1. Umgekehrt kann man aus jeder Wahrheitstafel die Funktion ablesen: Man verknüpft die Minterme aller Zeilen mit $f = 1$f = 1 durch ODER. Diese Form heißt kanonische disjunktive Normalform (KDNF); mehr dazu in Vorlesung 03.

Wenn die Wahrheitstafel eine Funktion eindeutig festlegt, kann man alle Funktionen mit gleich vielen Eingängen vollständig aufzählen. Für einen Eingang sind das vier, für zwei Eingänge sechzehn. Die nächsten beiden Folien ordnen sie und geben ihnen Namen.

> Und die Schaltfunktionen? Kann man diese auch systematisch darstellen?

Schaltfunktionen mit einem Eingang

Mit einem Eingang hat die Tafel zwei Zeilen, also gibt es zwei hoch zwei, gleich vier Funktionen. Zwei davon beachten den Eingang gar nicht und liefern konstant null oder konstant eins. Bleiben die Identität, die den Eingang unverändert durchreicht, und die Negation, die ihn umkehrt; als Gatter heißt sie Inverter.

Die möglichen 4 Kombinationen einer Schaltfunktion mit einem Eingang lassen sich wie folgt gliedern:

Eingang	Nullfunktion	Identität	Negation	Einsfunktion
$x = 0$x = 0	0	0	1	1
$x = 1$x = 1	0	1	0	1
	$f(x) = 0$ f(x) = 0	$f(x) = x$ f(x) = x	$f(x) = \bar{x}$ f(x) = x	$f(x) = 1$ f(x) = 1

Schaltfunktionen mit zwei Eingängen

Mit zwei Eingängen hat die Tafel vier Zeilen, also gibt es zwei hoch vier, gleich sechzehn Funktionen. Ihre Nummer ergibt sich aus der Ausgabespalte, gelesen von oben nach unten als Binärzahl. Beim Und steht dort null null null eins, das ist die Zahl eins, also f eins; beim Oder null eins eins eins, also f sieben.

Die möglichen 16 Kombinationen einer Schaltfunktion mit zwei Eingängen lassen sich wie folgt gliedern. Liest man die Ausgabespalte von oben nach unten als Binärzahl, erhält man ihre Nummer f_0 f0 bis f_{15}f15. Die vier Tabellen sind zum Nachschlagen gedacht – merken sollten Sie sich die Nummerierung und die Namen UND, ODER, XOR, NAND und NOR.

Eingang x	**Eingang y**	f_0**f0: Nullfunktion**	f_1**f1: Konjunktion (UND, AND)**	f_2**f2: Inhibition**
$x = 0$x = 0	$y = 0$y = 0	0	0	0
$x = 0$x = 0	$y = 1$y = 1	0	0	0
$x = 1$x = 1	$y = 0$y = 0	0	0	1
$x = 1$x = 1	$y = 1$y = 1	0	1	0
		$f(x, y) = 0$ f(x, y) = 0	$f(x, y) = x \cdot y$ f(x, y) = x · y	$f(x, y) = x \cdot \bar{y}$ f(x, y) = x · y

Eingang x	**Eingang y**	f_4**f4: Inhibition**	f_5**f5: Identität** y**y**	f_6**f6: Antivalenz (exklusives ODER, XOR,** $x \oplus$ **x ⊕ y)**
$x = 0$x = 0	$y = 0$y = 0	0	0	0
$x = 0$x = 0	$y = 1$y = 1	1	1	1
$x = 1$x = 1	$y = 0$y = 0	0	0	1
$x = 1$x = 1	$y = 1$y = 1	0	1	0
		$f(x, y) = \bar{x} \cdot y$ f(x, y) = x · y	$f(x, y) = y$ f(x, y) = y	$f(x, y) = x \cdot \bar{y} + \bar{x} \cdot$ f(x, y) = x · y + x · y

Sechs der sechzehn Funktionen sind alte Bekannte: die beiden Konstanten sowie Identität und Negation von nur einem Eingang. Die übrigen zehn hängen wirklich von beiden Eingängen ab, darunter Und, Oder, das exklusive Oder, NAND und NOR. Achten Sie auf das Muster der Nummern: f fünfzehn minus i ist immer die Negation von f i, so ist NAND mit der Nummer vierzehn die Negation von Und mit der Nummer eins.

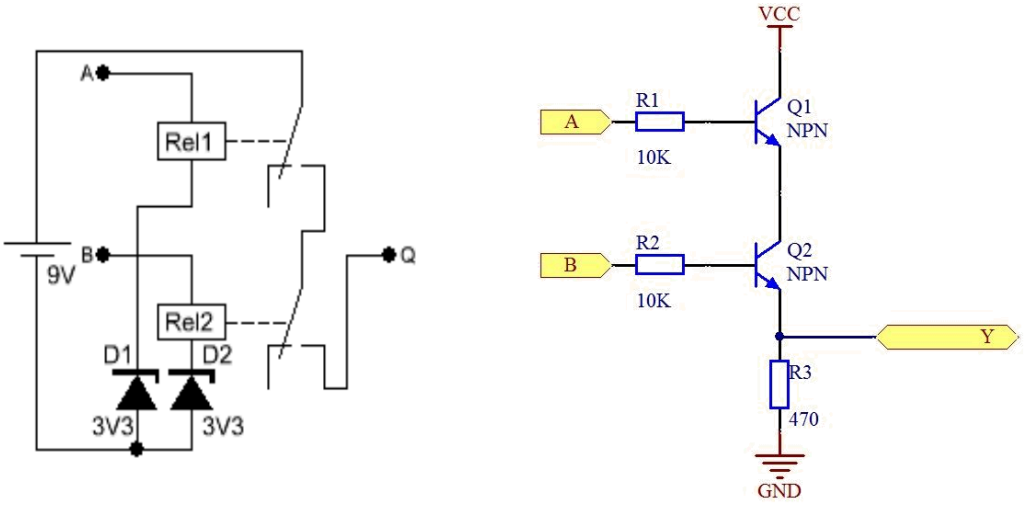
Eingang x	Eingang y	f_8f8: negiertes ODER (NOR, Peirce-Funktion)	f_9f9: Äquivalenz (XNOR)	f_{10}f10: Negation
$x = 0$x = 0	$y = 0$y = 0	1	1	1
$x = 0$x = 0	$y = 1$y = 1	0	0	0
$x = 1$x = 1	$y = 0$y = 0	0	0	1
$x = 1$x = 1	$y = 1$y = 1	0	1	0
		$f(x, y) = x \barwedge y$ f(x, y) = x + y	$f(x, y) = x \cdot y + \bar{x} \cdot \bar{y}$ f(x, y) = x · y + x · y	$f(x, y) = \bar{y}$ f(x, y) = y

Eingang x	Eingang y	f_{12}f12: Negation $\bar{x}$x	f_{13}f13: Implikation $x \to y$x → y	f_{14}f14: negiertes UND (NAND, Sheffer-Funktion)
$x = 0$x = 0	$y = 0$y = 0	1	1	1
$x = 0$x = 0	$y = 1$y = 1	1	1	1
$x = 1$x = 1	$y = 0$y = 0	0	0	1
$x = 1$x = 1	$y = 1$y = 1	0	1	0
		$f(x, y) = \bar{x}$ f(x, y) = x	$f(x, y) = \bar{x} + y$ f(x, y) = x + y	$f(x, y) = x \barwedge y$ f(x, y) = x · y

Exkurs: Technische Realisierung

Ein Gatter ist die technische Umsetzung einer Booleschen Funktion. Die beiden Bilder zeigen dieselbe Und-Funktion zweimal, einmal mit zwei Relais und einmal mit zwei Transistoren, jeweils in Reihe geschaltet. Das Prinzip ist die Reihenschaltung aus der Motivation, nur wird jeder Schalter jetzt elektrisch betätigt statt von Hand.

Ein Gatter ist eine (elektrotechnische) „Black Box“ mit einem, zwei oder mehreren Eingängen $A, B, C, ... \in \{0, 1\}$A, B, C, ... ∈ {0, 1} und genau einem Ausgang $Y \in \{0, 1\}$Y ∈ {0, 1} zur Realisierung einer Funktion $Y = f(A, B, C, ...)$Y = f(A, B, C, ...)



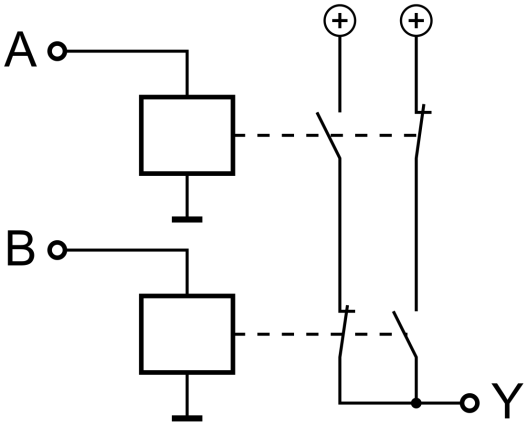
Wir gehen bei der Frage der Schaltnetze in Vorlesung 04 nochmals auf die technische Realisierung ein.

Das exklusive Oder entsteht nicht einfach durch Reihe oder Parallelschaltung. Der Ausgang soll eins sein, wenn genau einer der beiden Eingänge eins ist, nicht aber, wenn beide es sind. Ein Hinweis für Ihre Überlegung: Ein Relais kann einen Kontakt auch öffnen, statt ihn zu schließen.

> Und das exklusive ODER, hätten Sie eine Idee?

a**a**	b**b**	y**y**
0	0	0
0	1	1
1	0	1
1	1	0

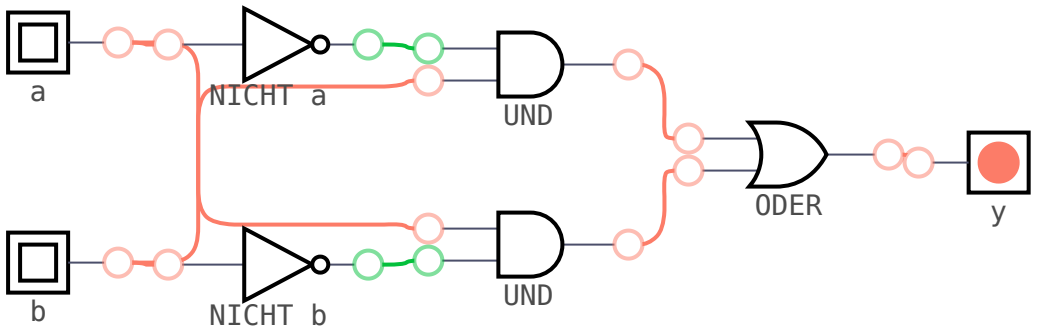
Jedes Relais betätigt hier zwei Kontakte gleichzeitig: einen Schließer, der bei Ansteuerung schließt, und einen Öffner, der dabei öffnet. Ein Strompfad führt über den Schließer von a und den Öffner von b, er leitet also bei a und nicht b. Der andere Pfad leitet entsprechend bei nicht a und b. Beide Pfade führen parallel zum Ausgang, und genau das ist die Formel des exklusiven Oder.



Realisierung einer XOR Funktion mit Relais Autor: ‚Rk sTEk', XOR in Relais-Logik - berichtigte Version, https://de.m.wikipedia.org/wiki/Datei:Relay_xor.svg

Dieselbe Funktion lässt sich aus Grundgattern aufbauen, genau nach der Formel nicht a und b, oder a und nicht b. Jedes Und-Gatter erkennt einen der beiden Fälle, in denen die Eingänge verschieden sind, das Oder-Gatter fasst sie zusammen. Schalten Sie die Eingänge a und b in allen vier Kombinationen und vergleichen Sie die Lampe mit der Wahrheitstafel.

Dieselbe Funktion aus Gattern, $y = \bar{a}b + a\bar{b}$y = ab + ab. Probieren Sie alle vier Eingangskombinationen aus:



Exkurs: Schaltertechnik und Pegel in der Praxis

Zum Selbststudium – in der Vorlesung nur kurz.

Die Tabelle vergleicht zwei Techniken, mit denen man Schalter baut: das Relais wie in der Z3 und die CMOS-Technik, in der heute praktisch alle Prozessoren und Mikrocontroller gebaut sind. Dazwischen lagen Röhre und bipolarer Transistor. CMOS-Schalter sind schneller, kleiner und sparsamer; galvanisch getrennt ist nur das Relais. Für Ihre Praxis wichtig ist die Zeile zur Störempfindlichkeit: Ein offener CMOS-Eingang hat keinen definierten Pegel, deshalb braucht ein Taster einen Pull-up-Widerstand.

Kriterium / Technologie	Mechanisches Relais	**CMOS (alle heutigen Prozessoren und Mikrocontroller, z. B. ATmega328P)**
Funktionsprinzip	Mechanischer Schalter mit elektromagnetischer Betätigung	MOSFET-Gatter mit hochohmigem Gateeingang
Schaltzeit	5–20 ms	< 10 ns (intern)
Stromverbrauch (aktiv)	100–500 mW	pro Gatter nur beim Umschalten; ATmega328P bei 16 MHz und 5 V gesamt ca. 50 mW
Eingangswiderstand	< 1 kΩ (Spule)	sehr hoch, Leckstrom ≤ 1 µA
Lebensdauer (Schaltzyklen)	10^5 – 10^7	unbegrenzt (Halbleiter)
Größe	cm-Bereich	µm-Bereich (integriert)
Isolation	Galvanisch getrennt	Keine galvanische Trennung
Empfindlichkeit gegenüber Störungen	Gering	Hoch, wenn der Eingang offen ist
Kosten	Hoch (0,5 – 2 €)	Niedrig (Millionen Gatter pro Chip)
Typische Anwendung	Schaltsignale in Industrie oder Leistungselektronik	Prozessoren, Mikrocontroller, Logikbausteine

Hochohmig heißt: Der Eingang nimmt praktisch keinen Strom auf. Ist er mit nichts verbunden, „schwebt“ seine Spannung und fängt jede Störung ein. Deshalb braucht ein Taster am ATmega328P einen Pull-up-Widerstand (Vorlesung 00): Er legt den offenen Eingang definiert auf High.

Galvanische Trennung bedeutet, dass **kein direkter elektrischer Leiter** zwischen zwei Schaltungsteilen besteht. Die Information wird über andere physikalische Effekte übertragen, z. B.: Licht, Magnetfelder oder kapazitive Kopplung.

Ein Relais schaltet man nie direkt an einen Pin des Mikrocontrollers: Die Spule braucht mehr Strom, als ein Pin liefert, und erzeugt beim Abschalten eine Spannungsspitze. Dazwischen gehören ein Transistor und eine Freilaufdiode. Außerdem prellen die Kontakte beim Schließen.

TTL-Bausteine (74LS…) werden kaum noch neu eingesetzt; ein NAND- oder NOR-Baustein von heute ist CMOS (74HC…, 74LVC…). Die TTL-Pegel leben aber als Schnittstellenstandard weiter: 74HCT-Bausteine haben TTL-Eingangsschwellen und passen so 3,3-V-Signale an 5-V-Logik an, und „TTL-seriell“ heißt die serielle Schnittstelle mit Logikpegeln, über die der ATmega328P auf dem Arduino UNO R3 mit dessen USB-Wandler-Chip spricht.

Und wenn ein Signal trotzdem im verbotenen Bereich landet? Dann hilft keine Logik, sondern nur die Schaltung davor: ein Pegelwandler oder Pull-up-Widerstand, der Schmitt-Trigger im Eingang, der mit zwei Schwellen ein Hin- und Herspringen verhindert, oder der Analog-Digital-Wandler, wenn das Signal ohnehin analog ist.

Und wenn der Pegel nicht passt? Digitale Schaltungen lösen das nicht mit Logik, sondern davor:

- **Pegel sicherstellen:** Passt ein Ausgang nicht zum Eingang, gleicht ein **Pegelwandler** an. Ein 3,3-V-Sensor am ATmega328P mit 5 V liegt mit seinem High-Pegel nur knapp über V_{IH}VIH = 3,0 V – der Störabstand schrumpft auf wenige Zehntel Volt. Kritischer ist die Gegenrichtung: 5 V vom ATmega328P an einem 3,3-V-Eingang können diesen beschädigen. Ein offener Eingang bekommt einen Pull-up- oder Pull-down-Widerstand; eingebaut hat der ATmega328P nur Pull-ups (`INPUT_PULLUP`), einen Pull-down schaltet man extern. Spannungen über V_{CC}VCC + 0,5 V oder unter −0,5 V beschädigen den Eingang; dagegen helfen Spannungsteiler oder Vorwiderstand.
- **Schmitt-Trigger:** Jeder digitale Eingang des ATmega328P hat zwei Schwellen, eine zum Einschalten und eine etwas tiefere zum Ausschalten (Hysterese; typische Werte zeigt das Datenblatt im Abschnitt „Pin Threshold and Hysteresis“). Ein langsames oder verrauschtes Signal kippt dadurch einmal, statt an einer einzigen Schwelle hin- und herzuspringen.
- **Analog einlesen:** Ein analoges Sensorsignal liest man mit dem Analog-Digital-Wandler ein (Vorlesung 13) und legt die Schwelle selbst in Software fest – am besten ebenfalls mit Hysterese.

Auch jede Flanke durchläuft den verbotenen Bereich. Tastet ein Speicherglied genau in diesem Moment ab, kann es kurz zwischen 0 und 1 hängen bleiben (**Metastabilität**, Vorlesung 07).

Und im ATmega328P?

Boolesche Funktionen schreiben Sie in C als Bedingungen. Das folgende Programm läuft auf einem simulierten ATmega328P. Es gibt zuerst die Wahrheitstafel aus und steuert dann eine Presse mit Zweihandbedienung.

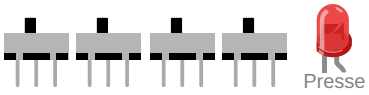
In C verknüpfen die logischen Operatoren `&&` (UND), `||` (ODER) und `!` (NICHT) Wahrheitswerte; `0` bedeutet falsch, jeder andere Wert wahr. Eine Presse mit Zweihandbedienung darf nur auslösen, wenn beide Taster gedrückt sind und der Not-Aus nicht betätigt ist:

$$f = \text{links} \cdot \text{rechts} \cdot \overline{\text{notAus}}$$

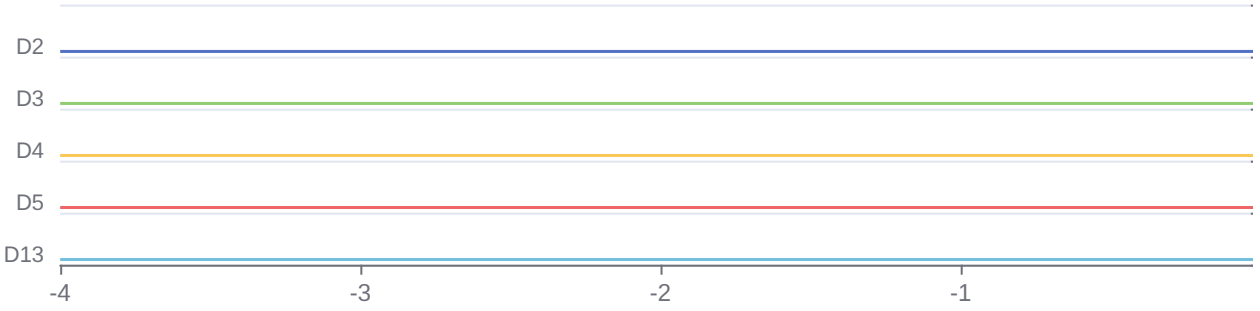
f = links · rechts · notAus

Das Beispiel ist bewusst vereinfacht. Eine echte Zweihandschaltung verlangt zusätzlich, dass beide Taster innerhalb von 0,5 s gedrückt werden – dafür muss sich die Schaltung etwas merken und die Zeit messen; sie ist dann keine reine Schaltfunktion mehr, sondern ein Schaltwerk (Vorlesung 06 und 07). Ein echter Not-Aus ist außerdem ein **Öffner**: Gedrückt oder bei Kabelbruch fällt das Signal weg, und die Maschine steht. Er wirkt nie über ungeprüfte Software, sondern über eine fest verdrahtete oder zertifizierte Sicherheitsschaltung. Im Simulator liefern die Schiebeschalter dagegen direkt High oder Low; am echten Board würden Sie die Taster gegen GND mit `INPUT_PULLUP` anschließen, dann bedeutet LOW „gedrückt“.

Starten Sie das Programm. Weil sich mit der Maus nicht zwei Taster gleichzeitig drücken lassen, stehen hier Schiebeschalter für die Taster: von links nach rechts der linke Taster (Pin 2), der rechte Taster (Pin 3), der Not-Aus (Pin 4) und der Schalter „Gitter offen“ (Pin 5) für die Aufgabe unten; die LED an Pin 13 zeigt, ob die Presse auslöst. Im Diagramm sehen Sie die Pegel der Pins.



links, rechts, Not-Aus, Gitter und Presse



Zweihandbedienung.ino

```
void setup() {
  Serial.begin(115200);
  // Simulator: Schiebeschalter liefern direkt HIGH/LOW.
  // Am echten Board: INPUT_PULLUP, dann bedeutet LOW "gedrückt".
  pinMode(2, INPUT);    // Schalter links
  pinMode(3, INPUT);    // Schalter rechts
  pinMode(4, INPUT);    // Not-Aus
  pinMode(5, INPUT);    // Gitter offen (für die Aufgabe)
  pinMode(13, OUTPUT);  // LED: Presse löst aus

  // Wahrheitstafel und Probe der De Morgan'schen Regel
  Serial.println("a b | a&&b a||b !(a||b) !a&&!b");
  for (int a = 0; a <= 1; a++) {
    for (int b = 0; b <= 1; b++) {
      Serial.print(a); Serial.print(" "); Serial.print(b); Serial.pri
          ");
      Serial.print(a && b);          Serial.print("    ");
      Serial.print(a || b);          Serial.print("      ");
      Serial.print(!(a || b));       Serial.print("       ");
      Serial.println(!a && !b);
    }
  }
}

void loop() {
  bool links  = digitalRead(2);
  bool rechts = digitalRead(3);
  bool notAus = digitalRead(4);
  bool gitter = digitalRead(5);   // noch nicht verwendet

  // Heute wichtig ist nur diese Bedingung:
  digitalWrite(13, links && rechts && !notAus);
  delay(20);
}
```

```
Sketch uses 2554 bytes (7%) of program storage space. Maximum is 32256
bytes.
Global variables use 232 bytes (11%) of dynamic memory, leaving 1816
bytes for local variables. Maximum is 2048 bytes.


a b | a&&b a||b !(a||b) !a&&!b
0 0 |   0    0      1       1
0 1 |   0    1      0       0
1 0 |   0    1      0       0
1 1 |   1    1      0       0
```

Achten Sie in der seriellen Ausgabe auf die letzten beiden Spalten. Die eine berechnet nicht, in Klammern a oder b, die andere nicht a und nicht b, und für alle vier Belegungen stimmen sie überein. Das Programm prüft die De Morgan'sche Regel also durch Ausprobieren aller Fälle, genau wie eine Wahrheitstafel.

Die letzten beiden Spalten der Ausgabe sind gleich – das ist die De Morgan'sche Regel $\overline{a + b} = \bar{a} \cdot \bar{b}$ a + b = a · b. Sie brauchen sie beim Umschreiben von Bedingungen.

Ein häufiger Fehler ist die Verwechslung der logischen Operatoren mit den bitweisen, die nur ein Zeichen haben. Die logischen fragen lediglich, ob ein Wert null ist oder nicht; die bitweisen verknüpfen jede Bitstelle einzeln. Für Bedingungen verwenden Sie die logischen Operatoren, die bitweisen brauchen wir erst, wenn wir einzelne Bits in Registern setzen.

Typische Fehlvorstellung

`&&` **ist nicht** `&`**.** In C gibt es zu jedem logischen Operator einen bitweisen Zwilling: `&` statt `&&`, `|` statt `||`, `~` statt `!`. Die bitweisen Operatoren verknüpfen jedes Bit zweier Zahlen einzeln. Solange nur 0 und 1 vorkommen, liefern `&` und `&&` sowie `|` und `||` dasselbe – sonst nicht: `2 && 1` ist wahr (1), denn beide Werte sind ungleich 0, aber `2 & 1` ergibt 0, denn `0b10` und `0b01` haben kein gemeinsames gesetztes Bit. Für `~` gilt das nie: `~` kippt alle Bits, `~1` ergibt −2 und ist damit wahr, `!1` dagegen 0. Außerdem wertet `&&` die rechte Seite gar nicht erst aus, wenn die linke schon falsch ist. Für Bedingungen verwenden Sie deshalb immer `&&`, `||` und `!`. Die bitweisen Operatoren brauchen wir in Vorlesung 12, um einzelne Bits in den Registern des ATmega328P zu setzen.

Probieren Sie es aus: Ändern Sie die Bedingung so, dass die Presse auch dann nicht auslöst, wenn der Schalter „Gitter offen“ (Pin 5) betätigt ist. Wie lautet die Funktion ff dann?

- ◯ `links && rechts && !notAus || !gitter`
- ◯ `links && rechts && !notAus && !gitter`
- ◯ `links && rechts && !(notAus && gitter)`

Die beiden Fragen üben De Morgan in C. Ein Nicht vor einer Klammer wandert hinein; dabei wird jede Variable negiert, und aus Oder wird Und und umgekehrt. Prüfen Sie Ihre Antwort im Zweifel mit einer kleinen Wahrheitstafel.

Welcher Ausdruck ist gleichwertig zu `!(a || b)`?

- ◯ `!a || !b`
- ◯ `!a && !b`
- ◯ `a && b`
- ◯ `!a && b`

Die Presse soll auslösen, wenn nicht gilt: „linker Taster losgelassen oder rechter Taster losgelassen“. Welche Bedingung ist das?

- ◯ `!links || !rechts`
- ◯ `links && rechts`
- ◯ `links || rechts`
- ◯ `!(links && rechts)`

Was Sie heute mitnehmen

Diese fünf Punkte fassen die Vorlesung zusammen. Die Fragen darunter gehören zu den Lernzielen vom Anfang: Wenn Sie sie beantworten und an einem Beispiel vorrechnen können, sind Sie gut auf die Minimierung in der nächsten Woche vorbereitet.

- **Digital heißt: Bereiche statt Werte.** Ein Eingang deutet jede Spannung als 0 oder 1; dazwischen liegt ein verbotener Bereich. Der **Störabstand** zwischen dem, was ein Ausgang liefert, und dem, was ein Eingang verlangt, macht Digitaltechnik robust.
- **Zwei Werte, drei Operationen, vier Axiome:** Mit $+$+, $\cdot$· und $\bar{a}$a sowie Kommutativität, Distributivität, neutralen Elementen und Komplement lassen sich alle weiteren Gesetze herleiten – Idempotenz, Absorption, **De Morgan**.
- **Dualitätsprinzip:** Vertauscht man UND mit ODER und 0 mit 1, wird aus jedem Gesetz wieder ein gültiges Gesetz.
- **Die Wahrheitstafel** beschreibt eine Boolesche Funktion eindeutig; Ausdrücke und Schaltnetze gibt es für dieselbe Funktion viele. Mit nn Eingängen hat sie 2^n2n Zeilen, und es gibt 2^{2^n}22n verschiedene Funktionen.
- **In C** schreiben Sie Boolesche Funktionen als Bedingungen mit `&&`, `||` und `!` – nicht mit den bitweisen Operatoren `&`, `|` und `~`.

Prüfen Sie sich:

- Was unterscheidet ein Digitalsignal von einem Analogsignal, und was gibt der Störabstand an?
- Nennen Sie die Axiome der Booleschen Algebra.
- Erläutern Sie das Dualitätsprinzip der Booleschen Algebra.
- Geben Sie die De Morgan'schen Regeln wieder.
- Wie viele Zeilen hat die Wahrheitstafel einer Schaltfunktion mit nn Eingängen, und wie viele Schaltfunktionen mit zwei Eingängen gibt es?
- Nennen Sie zwei Beispiele, wie eine Schaltfunktion technisch umgesetzt werden kann.
- Wie schreiben Sie eine Boolesche Funktion als Bedingung in C, und wie hilft die De Morgan'sche Regel beim Umformen?

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam über das LiaScript-Klassenzimmer; nach der Vorlesung werden die Begriffe dieser Woche in der Karte freigeschaltet.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe ...

Wo begegnet Ihnen das, was heute dran war – in Ihrem Studiengang oder in Geräten Ihres Alltags?

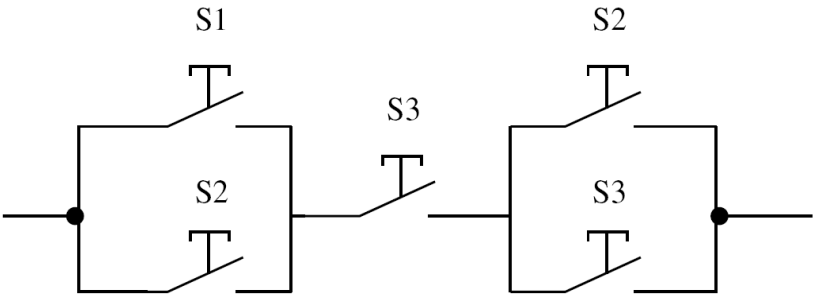
Texteingabe ...

Hausaufgaben

Bis zum Beginn der Übungen Ende November ersetzt die Hausaufgabe die Übung. Planen Sie 30–45 Minuten ein. Teil A prüfen Sie direkt hier, Teil B besprechen wir zu Beginn der nächsten Vorlesung.

Teil A – mit Selbstkontrolle

Aufgabe 1: Geben Sie für die folgende Schaltung eine Boolesche Funktion an und stellen Sie die Wahrheitstafel auf. Ein geschlossener Schalter entspricht 1.



In wie vielen Zeilen der Wahrheitstafel ist $f = 1$f = 1?

Welcher Ausdruck ist die vereinfachte Funktion?

- $S_1 + S_2 + S_3$S1 + S2 + S3
- $S_3(S_1 + S_2)$S3(S1 + S2)
- $S_1 S_2 S_3$S1S2S3
- S_3S3

Aufgabe 2: Entwerfen Sie nur mit den Gattern UND, ODER und NICHT Schaltnetze, die PP und QQ aus den Eingängen XX, YY und ZZ erzeugen. Lösen Sie dazu zuerst das XOR mit seiner Definition auf. Vergleichen Sie PP und QQ mit Wahrheitstafeln.

$$\begin{aligned} P &= \left(X + \bar{Y}\right)\ (Y \oplus Z) \\ Q &= \bar{Y}Z + XY\bar{Z} \end{aligned}$$

P Q = (X + Y) (Y ⊕ Z) = Y Z + XY Z

Welche Beziehung besteht zwischen PP und QQ?

- ◯ $P = Q$P = Q für alle Eingaben
- ◯ $P = \bar{Q}$P = Q für alle Eingaben
- ◯ PP und QQ unterscheiden sich in genau einer Zeile
- ◯ es gibt keinen einfachen Zusammenhang

Teil B – Besprechung in der nächsten Vorlesung

Aufgabe 3: Lesen Sie das Datenblatt des UND-Gatters SN74LVC2G08 von Texas Instruments:

- Wie groß ist die maximale Verzögerung (*propagation delay* t_{pd}tpd), mit der der Ausgang dem Eingang folgt, bei einer Versorgungsspannung von 3,3 V?
- Was bedeuten die Kreuze in der Wahrheitstafel (*Function table*)? (Der Begriff dafür kommt in Vorlesung 03.)
- Können Sie mit dem Gatter auch eine Negation des Eingangssignals realisieren? Begründen Sie mit der Wahrheitstafel.

Nächste Vorlesung

Am **Mittwoch, 04.11.**, geht es um Minimierung: Mit dem KV-Diagramm finden Sie die kürzeste Form einer Schaltfunktion, ohne lange Umformungsketten.

[ATmega] Microchip Technology: ATmega48A/PA/88A/PA/168A/PA/328/P megaAVR® Data Sheet, DS40002061B, 2020, Table 30-1 „Common DC characteristics“, Link